

# P1 - Uso de Smartphones y Riesgo de Adiccion

**Hiter Frank Rubio Peralta**

Alumno

**Jossep Jorge Froilan Choroco Neyra**

Alumno

**Joel David De la Cruz Cardenas**

Alumno

**Rodrigo Octavio Lucatoni Flores Ruiz**

Alumno

**Victor Andres Diaz Garcia**

Alumno

Curso:

Introduccion a Ciencia de Datos e Inteligencia Artificial

**22 de Septiembre del 2026**

## Tabla de contenidos

|                                                           |           |
|-----------------------------------------------------------|-----------|
| <b>1. Resumen</b>                                         | <b>3</b>  |
| <b>2. Planteamiento del problema</b>                      | <b>3</b>  |
| <b>3. Contexto del dataset</b>                            | <b>3</b>  |
| 3.1. Fuente y unidad de análisis . . . . .                | 3         |
| 3.2. Diccionario de variables . . . . .                   | 3         |
| <b>4. Carga y exploración inicial</b>                     | <b>4</b>  |
| 4.1. Dimensiones y tipos de variables . . . . .           | 5         |
| 4.2. Calidad de datos . . . . .                           | 6         |
| <b>5. Preprocesamiento y decisiones de data wrangling</b> | <b>7</b>  |
| 5.1. Revisión de fuga de información . . . . .            | 7         |
| 5.2. Variable derivada para la comunicación . . . . .     | 9         |
| <b>6. Análisis exploratorio de datos</b>                  | <b>9</b>  |
| 6.1. Análisis univariado . . . . .                        | 9         |
| 6.2. Análisis multivariado . . . . .                      | 22        |
| <b>7. Conclusiones de P1</b>                              | <b>30</b> |
| <b>8. Limitaciones y consideraciones éticas</b>           | <b>31</b> |

## Listado de Tablas

|                                                                                      |   |
|--------------------------------------------------------------------------------------|---|
| 1.    Primeras 10 filas del dataset (ya con nombres de columna en español) . . . . . | 4 |
|--------------------------------------------------------------------------------------|---|

## Listado de Figuras

### 1. Resumen

*Instrucciones para esta sección: este resumen se escribe **al final**, cuando ya se tengan los resultados de la exploración de datos y las conclusiones de P1. Debe tener entre 150 y 200 palabras e incluir, en este orden: (1) qué dataset se usó y cuántos registros/columnas tiene, (2) el objetivo del análisis (qué se busca entender sobre `etiqueta_de_adiccion`), (3) las dos o tres decisiones de limpieza más importantes que se tomaron y por qué, (4) los dos o tres hallazgos más relevantes del EDA, citando cifras reales obtenidas del código (no inventadas ni copiadas de un borrador), y (5) una frase que conecte con lo que se hará en la P2. No copiar texto de otras secciones; debe ser una síntesis propia, escrita por el equipo.*

### 2. Planteamiento del problema

El uso intensivo de celulares puede influir en la salud integral de las personas al relacionarse con irregularidades en el sueño, estrés, dificultades en el ámbito académico y laboral. Además, la investigación requiere de una conceptualización alta: una alta frecuencia de uso no significa necesariamente a una condición clínica. Artículos como SABAS permiten estudiar síntomas de uso problemático, pero no sustituyen una evaluación profesional Csibi et al. (2018). La literatura también ayuda con asociaciones entre el uso del smartphone y el desempeño académico, aunque con limitaciones claras para atribuir causalidad Amez y Baert (2020).

Para este proyecto, se formula la siguiente pregunta de análisis:

¿Qué patrones de uso, descanso y contexto personal están asociados con la etiqueta de adicción en el conjunto de datos `smartphone-use-prediction`?

### 3. Contexto del dataset

#### 3.1. Fuente y unidad de análisis

El archivo `Smartphone_Usage_And_Addiction_Analysis_7500_Rows.csv` fue proporcionado para el proyecto y corresponde al dataset publicado como *Smartphone Usage and Addiction Analysis* en Kaggle Nusrat (s.f.). Cada fila representa a un usuario y contiene indicadores demográficos, de comportamiento digital, descanso, estrés e impacto percibido en actividades académicas o laborales.

La unidad de análisis es el usuario. El dataset incluye 7,500 registros y 16 columnas originales: dos identificadores, nueve variables numéricas, cuatro variables categóricas y la etiqueta binaria objetivo.

#### 3.2. Diccionario de variables

- **Identificadores:** `ID_transaccion` y `ID_usuario` son variables de texto. Se usan para verificar unicidad, pero no aportan significado analítico y se excluirán como predictores.
- **Demografía:** `edad` (numérica) y `genero` (categórica) caracterizan la muestra.
- **Uso del smartphone:** `horas_diarias_pantalla`, `horas_redes_sociales`, `horas_videojuegos`, `horas_trabajo_estudio`, `notificaciones_por_dia`, `aperturas_apps_por_dia` y `tiempo_en_pantalla_fin_de_semana` son variables numéricas que miden intensidad y tipo de uso.

- **Bienestar y contexto:** horas\_sueño es numérica; nivel\_estres y impacto\_academico/laboral son categóricas. Se emplean para explorar descanso e impacto funcional.
- **Severidad:** nivel\_adiccion es categórica ordinal. Se describe en P1, pero se excluye como predictor en P2 por fuga de información.
- **Objetivo:** etiqueta\_de\_adiccion es binaria: 0 significa *No adicción* y 1 significa *Con adicción*.

La variable nivel\_adiccion contiene una categorización de severidad (None, Mild, Moderate, Severe). Por su definición operativa, puede estar construida directamente a partir de la etiqueta final; por ello se audita explícitamente antes de considerar su uso en P2.

## 4. Carga y exploración inicial

```
# Cargamos el archivo csv
import kagglehub
path = kagglehub.dataset_download("jayjoshi37/smartphone-usage-and-addiction-prediction")
import os

# Ruta directa al archivo CSV combinando la ruta de descarga y el nombre del archivo
ruta_csv = os.path.join(path, "Smartphone_Usage_And_Addiction_Analysis_7500_Rows.csv")
df = pd.read_csv(ruta_csv)

# Obtenemos la lista de columnas
df.columns

# Renombramos todas las columnas para un uso natural
df = df.rename({'transaction_id': 'ID_transaccion' , 'user_id': 'ID_usuario' , 'age': 'edad'

# Ejemplo de tabla bien formada: sin print(), como última línea del chunk, con label + tb
df.head(6)
```

|   | ID_transaccion | ID_usuario | edad | genero | horas_diarias_pantalla | horas_redes_sociales | horas_videojuegos |
|---|----------------|------------|------|--------|------------------------|----------------------|-------------------|
| 0 | TXN00001       | U00001     | 21   | Male   | 3.23                   | 2.01                 | 0.89              |
| 1 | TXN00002       | U00002     | 24   | Other  | 5.09                   | 3.81                 | 2.24              |
| 2 | TXN00003       | U00003     | 31   | Other  | 6.06                   | 1.36                 | 3.83              |
| 3 | TXN00004       | U00004     | 32   | Other  | 7.83                   | 5.85                 | 1.51              |
| 4 | TXN00005       | U00005     | 25   | Male   | 9.96                   | 5.92                 | 3.42              |
| 5 | TXN00006       | U00006     | 26   | Male   | 9.32                   | 4.26                 | 0.29              |

```
<class 'pandas.DataFrame'>
RangeIndex: 7500 entries, 0 to 7499
```

Data columns (total 16 columns):

| #  | Column                           | Non-Null Count | Dtype   |
|----|----------------------------------|----------------|---------|
| 0  | ID_transaccion                   | 7500 non-null  | str     |
| 1  | ID_usuario                       | 7500 non-null  | str     |
| 2  | edad                             | 7500 non-null  | int64   |
| 3  | genero                           | 7500 non-null  | str     |
| 4  | horas_diarias_pantalla           | 7500 non-null  | float64 |
| 5  | horas_redes_sociales             | 7500 non-null  | float64 |
| 6  | horas_videojuegos                | 7500 non-null  | float64 |
| 7  | horas_trabajo/estudio            | 7500 non-null  | float64 |
| 8  | horas_sueño                      | 7500 non-null  | float64 |
| 9  | notificaciones_por_dia           | 7500 non-null  | int64   |
| 10 | aperturas_apps_por_dia           | 7500 non-null  | int64   |
| 11 | tiempo_en_pantalla_fin_de_semana | 7500 non-null  | float64 |
| 12 | nivel_estres                     | 7500 non-null  | str     |
| 13 | impacto_academico/laboral        | 7500 non-null  | str     |
| 14 | nivel_adiccion                   | 6681 non-null  | str     |
| 15 | etiqueta_de_adiccion             | 7500 non-null  | int64   |

dtypes: float64(6), int64(4), str(6)

memory usage: 937.6 KB

```
Index(['ID_transaccion', 'ID_usuario', 'edad', 'genero',
      'horas_diarias_pantalla', 'horas_redes_sociales', 'horas_videojuegos',
      'horas_trabajo/estudio', 'horas_sueño', 'notificaciones_por_dia',
      'aperturas_apps_por_dia', 'tiempo_en_pantalla_fin_de_semana',
      'nivel_estres', 'impacto_academico/laboral', 'nivel_adiccion',
      'etiqueta_de_adiccion'],
      dtype='str')
```

## 4.1. Dimensiones y tipos de variables

```
# TODO: mostrar df.shape y df.dtypes (o df.info()) para revisar dimensiones y tipos de va
df.shape
```

(7500, 16)

```
# TODO: mostrar df.shape y df.dtypes (o df.info()) para revisar dimensiones y tipos de va
df.dtypes
```

```
ID_transaccion      str
ID_usuario           str
edad                int64
genero              str
horas_diarias_pantalla float64
```

```

horas_redes_sociales      float64
horas_videojuegos         float64
horas_trabajo/estudio     float64
horas_sueño               float64
notificaciones_por_dia    int64
aperturas_apps_por_dia    int64
tiempo_en_pantalla_fin_de_semana float64
nivel_estres              str
impacto_academico/laboral str
nivel_adiccion            str
etiqueta_de_adiccion      int64
dtype: object

```

*Instrucciones para esta sección: comentar, con base en la salida real de `df.info()` / `df.dtypes`, cuántas filas y columnas tiene el dataset, qué columnas son numéricas y cuáles categóricas, y si algún tipo de dato no corresponde con lo esperado (por ejemplo, una variable numérica cargada como texto). No repetir de memoria las cifras ya mencionadas en “Contexto del dataset”; verificarlas con el código.*

### 4.2. Calidad de datos

```
# TODO: contar valores nulos por columna (df.isnull().sum()) y filas duplicadas (df.duplicated().sum())
```

```
# TODO: contar valores nulos por columna (df.isnull().sum()) y filas duplicadas (df.duplicated().sum())
df["nivel_adiccion"].isnull().sum()
#Notamos que hay 819 valores nulos en la columna 'nivel_adiccion'
```

```
np.int64(819)
```

*Instrucciones para esta sección: reportar cuántos valores faltantes se encontraron y en qué columna(s), y cuántas filas duplicadas existen. Explicar la causa probable de los faltantes (por ejemplo, si corresponden a una categoría real como “sin severidad registrada”) y justificar la decisión tomada: ¿se imputan, se eliminan o se excluye la columna? Si se excluye una variable por fuga de información, explicarlo aquí o remitir a la sección de fuga de información.*

```
# TODO: revisar rangos (df.describe()) de las variables numéricas y valores únicos de las categóricas
df.describe()
```

|       | edad        | horas_diarias_pantalla | horas_redes_sociales | horas_videojuegos | horas_trabajo/estudio |
|-------|-------------|------------------------|----------------------|-------------------|-----------------------|
| count | 7500.000000 | 7500.000000            | 7500.000000          | 7500.000000       | 7500.000000           |
| mean  | 26.568800   | 7.499912               | 3.273484             | 2.014183          | 3.242420              |
| std   | 5.197108    | 2.609188               | 1.585342             | 1.146039          | 1.600765              |
| min   | 18.000000   | 3.000000               | 0.500000             | 0.000000          | 0.500000              |
| 25 %  | 22.000000   | 5.220000               | 1.910000             | 1.020000          | 1.850000              |
| 50 %  | 27.000000   | 7.525000               | 3.270000             | 2.040000          | 3.230000              |
| 75 %  | 31.000000   | 9.810000               | 4.630000             | 2.990000          | 4.640000              |

|     | edad      | horas_diarias_pantalla | horas_redes_sociales | horas_videojuegos | horas_trabajo/estudio |
|-----|-----------|------------------------|----------------------|-------------------|-----------------------|
| max | 35.000000 | 12.000000              | 6.000000             | 4.000000          | 6.000000              |

Instrucciones para esta sección: comentar si los rangos observados en `df.describe()` son plausibles para lo que representa cada variable (por ejemplo, edades, horas de uso, horas de sueño). Señalar explícitamente cualquier valor fuera de rango lógico y qué se hizo con los identificadores (`ID_transaccion`, `ID_usuario`): ¿se conservan solo como referencia o se descartan del análisis?

## 5. Preprocesamiento y decisiones de data wrangling

Notamos que hay 819 valores nulos en la columna “nivel\_adiccion”, por lo que procederemos a analizar las filas que contengan valores nulos en esta columna

|    | ID_transaccion | ID_usuario | edad | genero | horas_diarias_pantalla | horas_redes_sociales | horas_videojuegos |
|----|----------------|------------|------|--------|------------------------|----------------------|-------------------|
| 0  | TXN00001       | U00001     | 21   | Male   | 3.23                   | 2.01                 | 0.89              |
| 1  | TXN00002       | U00002     | 24   | Other  | 5.09                   | 3.81                 | 2.24              |
| 8  | TXN00009       | U00009     | 21   | Other  | 4.38                   | 1.38                 | 2.72              |
| 11 | TXN00012       | U00012     | 27   | Other  | 5.16                   | 1.82                 | 2.27              |
| 13 | TXN00014       | U00014     | 32   | Male   | 4.09                   | 1.74                 | 1.35              |

### 5.1. Revisión de fuga de información

```
# TODO: construir una tabla cruzada (pd.crosstab) entre nivel_adiccion y etiqueta_de_adiccion

pd.crosstab(df["nivel_adiccion"], df["etiqueta_de_adiccion"])

# Notamos que la columna 'nivel_adiccion' o cae en 0 cuando es baja o cae en 1 cuando es
# para alguien de adicción media p alta, la información de 'etiqueta_de_adiccion' es la n
# puesto que queremos encontrar qué patrones de uso, descanso y contexto personal están a
```

| etiqueta_de_adiccion | 0    | 1    |
|----------------------|------|------|
| nivel_adiccion       |      |      |
| Mild                 | 1373 | 0    |
| Moderate             | 0    | 2874 |
| Severe               | 0    | 2434 |

Instrucciones para esta sección: interpretar la tabla cruzada anterior: ¿existe alguna categoría de `nivel_adiccion` que se asocie de forma perfecta o casi perfecta con una sola clase de `etiqueta_de_adiccion`? Si es así, explicar por qué eso constituye **fuga de información** y qué variables se decide excluir de  $X$  para la P2 (y por qué).

Notamos que la columna 'nivel\_adiccion' o cae en 0 cuando es baja o cae en 1 cuando es media o alta, no hay ninguna excepción para alguien de adicción media p alta, la información de 'etiqueta\_de\_adiccion' es la necesaria y objetivo para este análisis puesto que queremos encontrar qué patrones de uso, descanso y contexto personal están asociados con la etiqueta de adicción en el conjunto de datos que estamos abordando.

Esto permitirá no repetir información en el proceso de modelamiento, ya que 'nivel\_adiccion' ya revelaría que 'etiqueta\_de\_adiccion' es 1 o 0.

*Instrucciones para esta sección: explicar por qué excluir las variables identificadas en el punto anterior protege la validez de la evaluación del modelo. Indicar además qué método de codificación se planea usar en P2 para cada variable categórica (genero, impacto\_academico/laboral, nivel\_estres) y por qué (one-hot, binaria, ordinal, etc.), aclarando que cualquier transformación debe ajustarse solo con los datos de entrenamiento. Si se usa scikit-learn para esto, citar la librería (Pedregosa et al. (2011)).*

**Contrato de nombres para el resto del proyecto** (acordado por el equipo, para que el código de todos encaje sin romperse al juntarlo): a partir de aquí se crean y se reutilizan siempre las mismas variables:

```
# TODO: crear df_clean = df sin ID_transaccion, ID_usuario ni nivel_adiccion (por fuga de
# num_cols = lista de las 9 columnas numéricas (edad, horas_diarias_pantalla, horas_redes
#             horas_videojuegos, horas_trabajo/estudio, horas_sueño, notificaciones_por_dia
#             aperturas_apps_por_dia, tiempo_en_pantalla_fin_de_semana,)
# cat_cols = ["genero", "nivel_estres", "impacto_academico/laboral"]
```

### 5.1.1. Enriquecimiento de datos:

*Instrucciones para esta sección: quien complete esta parte debe dejar creadas exactamente las variables df\_clean, num\_cols y cat\_cols con esos nombres. Las secciones de EDA (más abajo) las dan por hechas y no deben volver a definir las.*

```
# TODO: crear df_clean = df sin ID_transaccion, ID_usuario ni nivel_adiccion (por fuga de
df_clean = df.drop(columns=["ID_transaccion", "ID_usuario", "nivel_adiccion"])
num_cols = ["edad", "horas_diarias_pantalla", "horas_redes_sociales",
"horas_videojuegos", "horas_trabajo/estudio", "horas_sueño", "notificaciones_por_dia",
"aperturas_apps_por_dia", "tiempo_en_pantalla_fin_de_semana"]
cat_cols = ["genero", "nivel_estres", "impacto_academico/laboral"]
```

|   | edad | genero | horas_diarias_pantalla | horas_redes_sociales | horas_videojuegos | horas_trabajo/estudio |
|---|------|--------|------------------------|----------------------|-------------------|-----------------------|
| 0 | 21   | Male   | 3.23                   | 2.01                 | 0.89              | 4.55                  |
| 1 | 24   | Other  | 5.09                   | 3.81                 | 2.24              | 4.44                  |
| 2 | 31   | Other  | 6.06                   | 1.36                 | 3.83              | 2.35                  |
| 3 | 32   | Other  | 7.83                   | 5.85                 | 1.51              | 3.54                  |
| 4 | 25   | Male   | 9.96                   | 5.92                 | 3.42              | 5.27                  |

|      | edad | genero | horas_diarias_pantalla | horas_redes_sociales | horas_videojuegos | horas_trabajo/estud |
|------|------|--------|------------------------|----------------------|-------------------|---------------------|
| 0    | 21   | Male   | 3.23                   | 2.01                 | 0.89              | 4.55                |
| 1    | 24   | Other  | 5.09                   | 3.81                 | 2.24              | 4.44                |
| 2    | 31   | Other  | 6.06                   | 1.36                 | 3.83              | 2.35                |
| 3    | 32   | Other  | 7.83                   | 5.85                 | 1.51              | 3.54                |
| 4    | 25   | Male   | 9.96                   | 5.92                 | 3.42              | 5.27                |
| ...  | ...  | ...    | ...                    | ...                  | ...               | ...                 |
| 7495 | 26   | Other  | 9.85                   | 1.75                 | 3.13              | 3.49                |
| 7496 | 35   | Male   | 5.67                   | 2.33                 | 2.76              | 5.90                |
| 7497 | 22   | Female | 9.99                   | 3.61                 | 1.09              | 1.16                |
| 7498 | 23   | Male   | 8.74                   | 1.59                 | 0.07              | 4.64                |
| 7499 | 27   | Female | 10.04                  | 5.40                 | 0.73              | 5.45                |

### 5.2. Variable derivada para la comunicación

```
# TODO: agregar a df_clean una columna nueva (por ejemplo, estado_adiccion) que traduzca

df_clean["estado_adiccion"] = df_clean["etiqueta_de_adiccion"].replace({1:"Adicto", 0 :
df_clean["estado_adiccion"].value_counts()
```

```
estado_adiccion
Adicto          5308
No adicto       2192
Name: count, dtype: int64
```

Instrucciones para esta sección: confirmar que la nueva variable de etiquetas legibles no introduce ninguna información distinta a `etiqueta_de_adiccion` (es solo una versión “para humanos” de la misma columna) y aclarar que para el modelamiento en P2 se seguirá usando la versión numérica original.

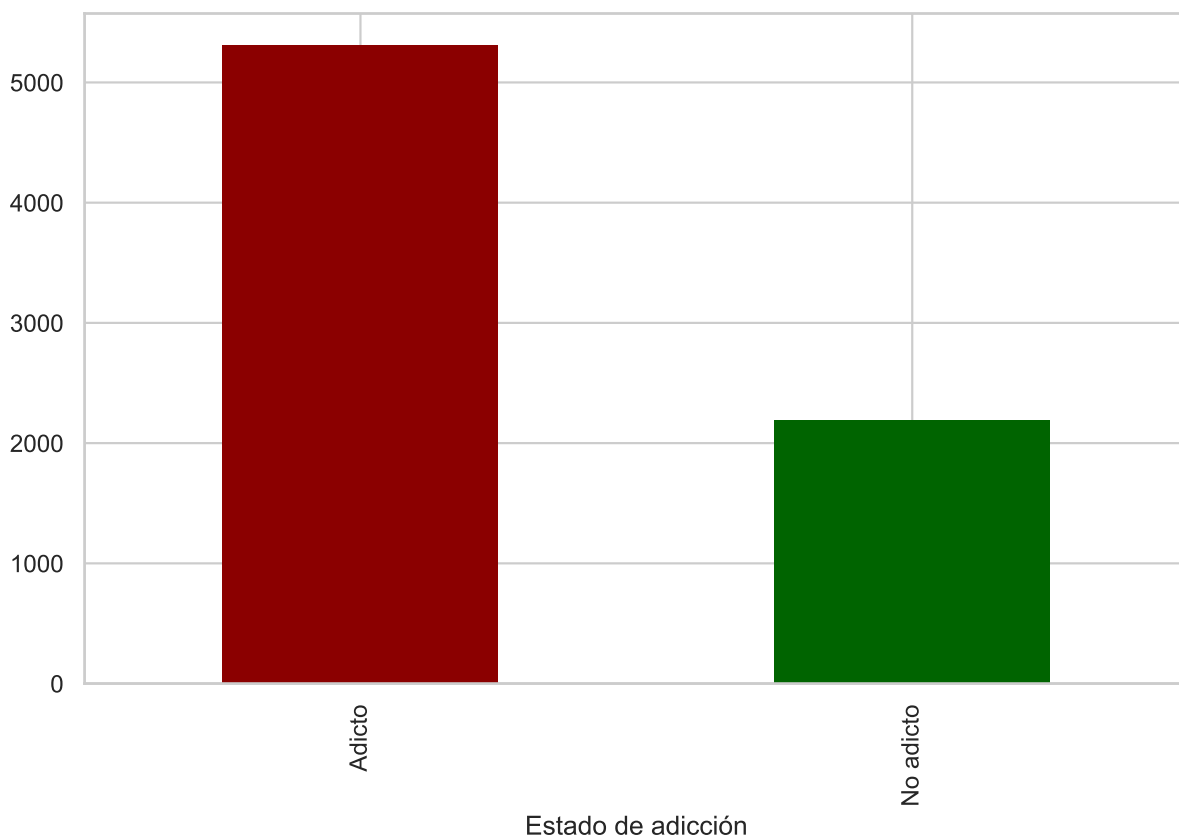
## 6. Análisis exploratorio de datos

### 6.1. Análisis univariado

```
# TODO: graficar la distribución de etiqueta_de_adiccion (conteo y/o porcentaje por clase
#sns.barplot(data=df_clean, x= "estado_adiccion", y = )
counts = df_clean["estado_adiccion"].value_counts()

counts.plot(kind="bar", color=['darkred','darkgreen'], edgecolor="none")

plt.xlabel("Estado de adicción")
plt.ylabel(None)
plt.show()
```

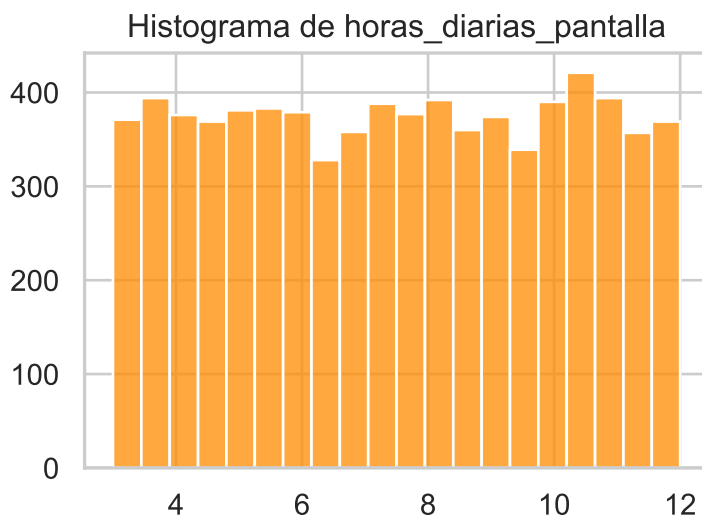
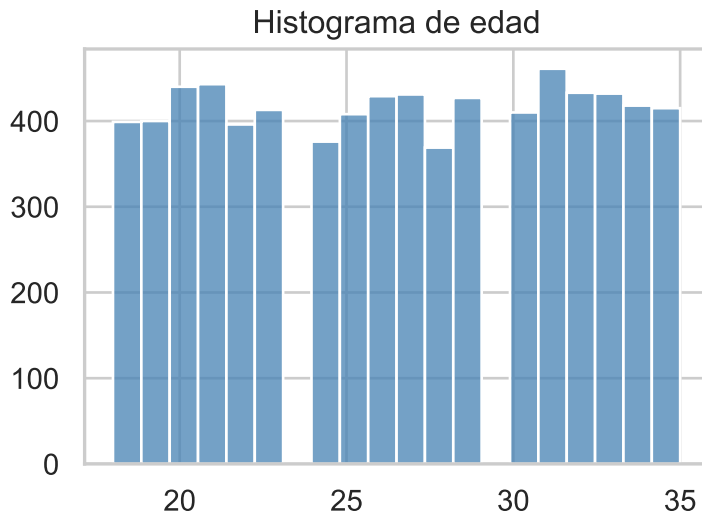


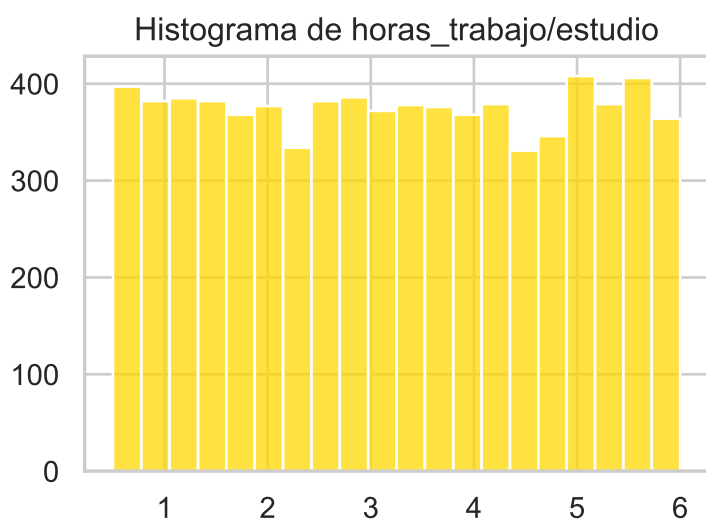
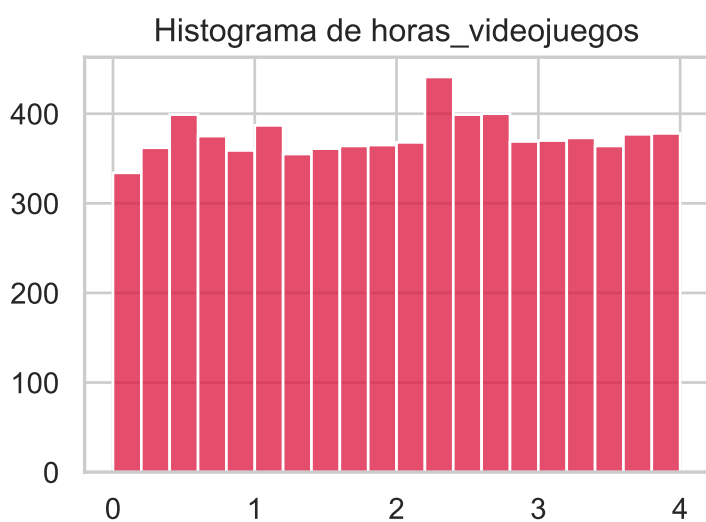
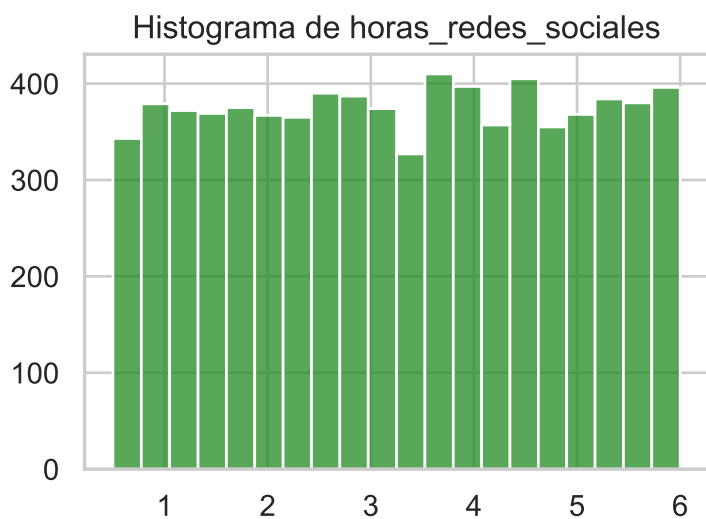
*Instrucciones para esta sección: reportar cuántos registros hay en cada clase de etiqueta\_de\_adiccion (conteo y porcentaje) y comentar si existe desbalance de clases. Si lo hay, explicar qué implicancia tiene para la partición entrenamiento/prueba y las métricas que se deberán usar en P2 (por ejemplo, estratificación, F1, recall).*

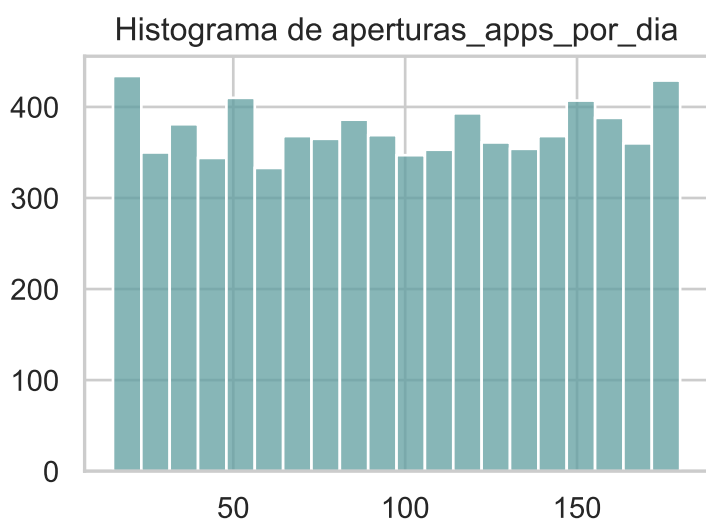
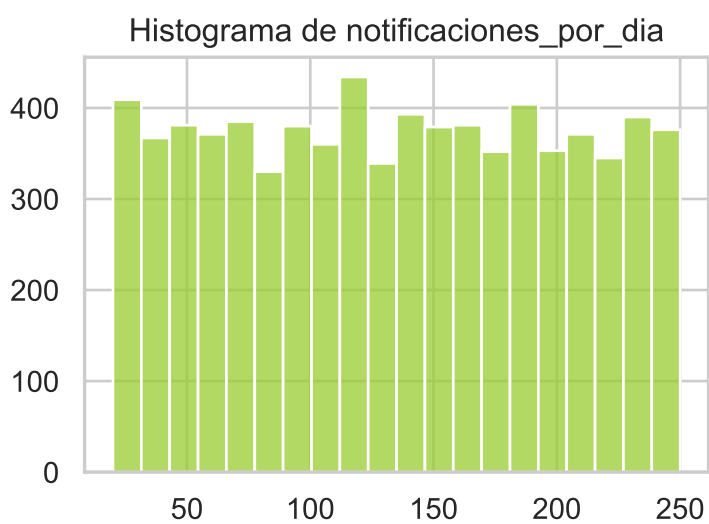
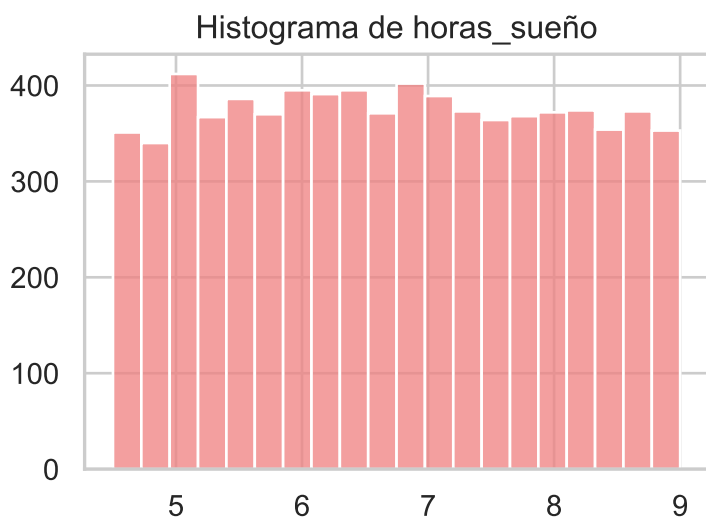
```
# TODO: graficar un histograma por cada variable numérica (usar num_cols).
# usa las columnas que ya tienes cargadas en la sesión

#crear una variable j que pase por una lista que tenga los nombres bien escritos de los
colores = ['steelblue','darkorange','forestgreen','crimson','gold','lightcoral','yellowgreen',
'powderblue','lightgreen','lightgray','plum']
contador = 0
for col in num_cols:
    plt.figure(figsize=(4, 3))
    sns.histplot(
        data=df_clean,
        x=col,
        bins=20,
        kde=False,
        color= colores[contador]
    )
    plt.title(f"Histograma de {col}")
```

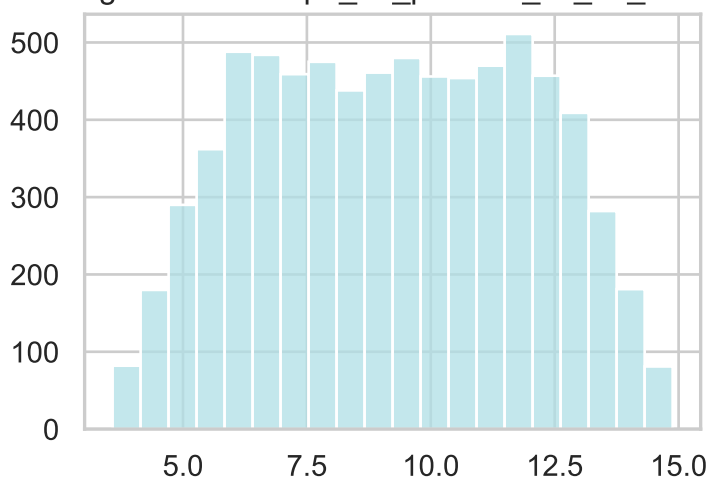
```
plt.xlabel(None)
plt.ylabel(None)
plt.tight_layout() #es una función de matplotlib que ajusta automáticamente los márgenes
plt.show()
contador +=1
```



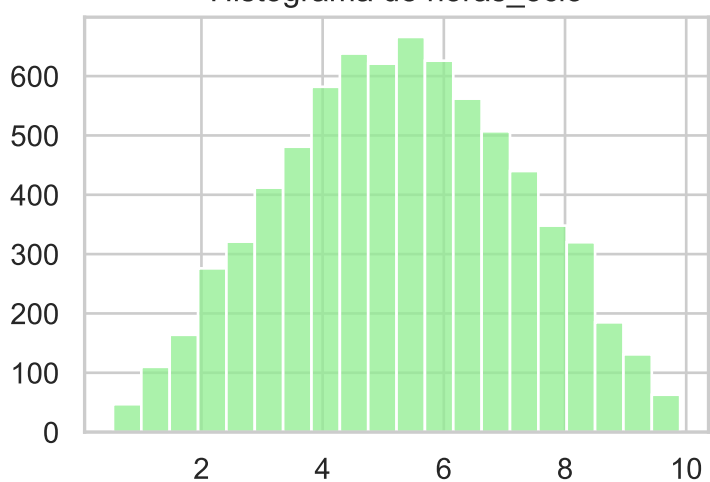




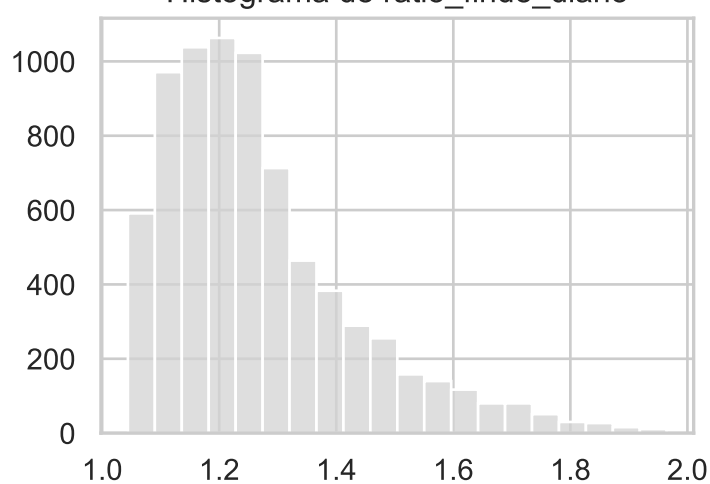
Histograma de tiempo\_en\_pantalla\_fin\_de\_semana

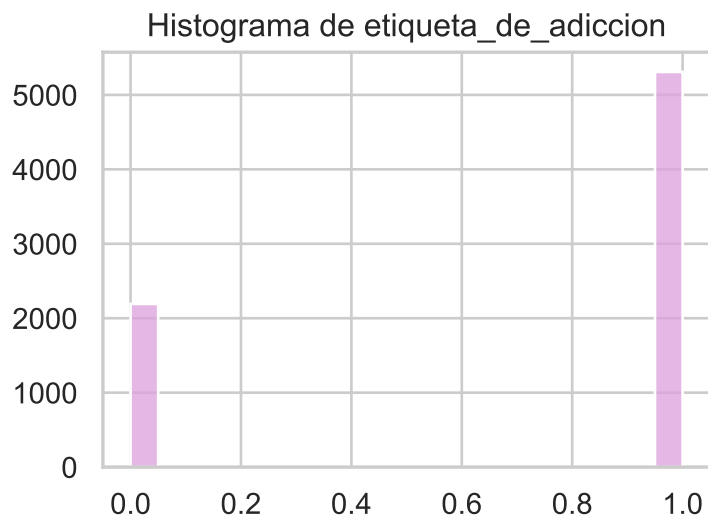


Histograma de horas\_ocio

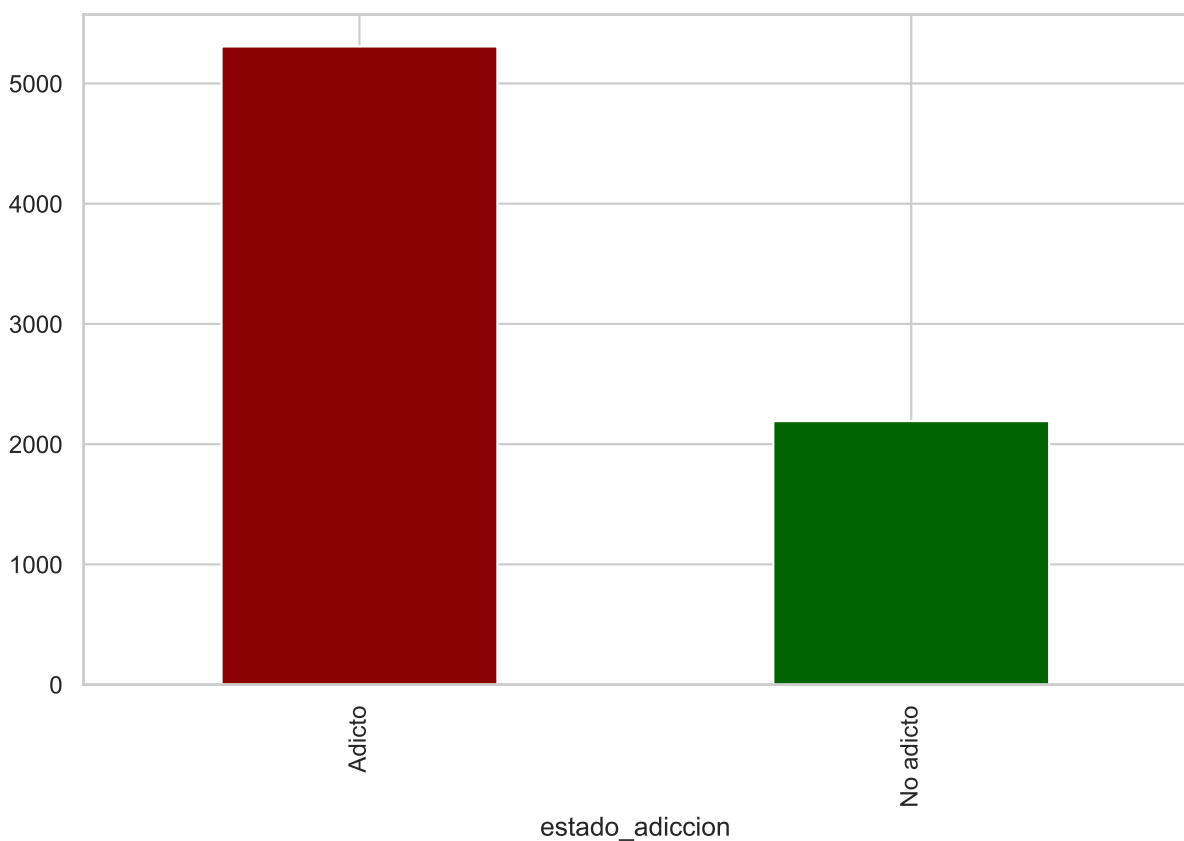


Histograma de ratio\_finde\_diario





```
# TODO: graficar un histograma por cada variable numérica (usar num_cols).
df_clean['estado_adiccion'].value_counts().plot(kind='bar',color=['darkred','darkgreen'])
plt.show() #usamos la variable comunicativa de 'etiqueta_de_adiccion'
```

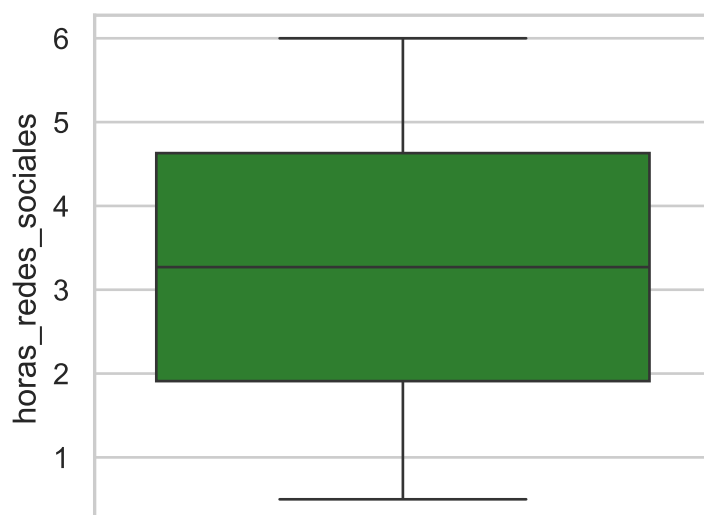
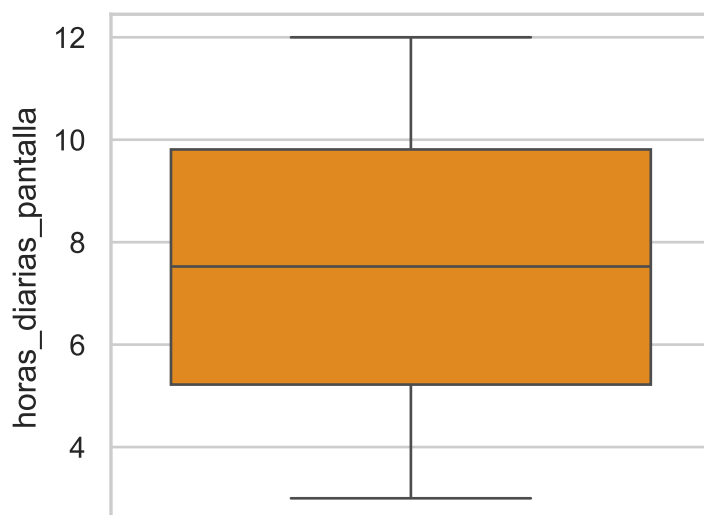
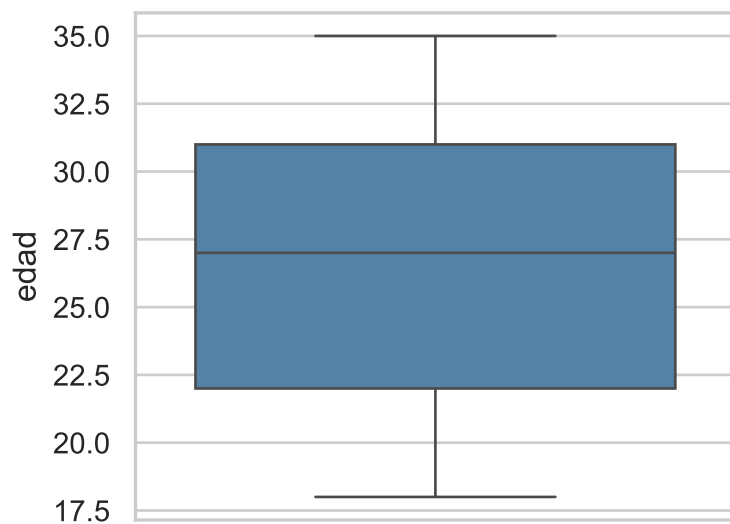


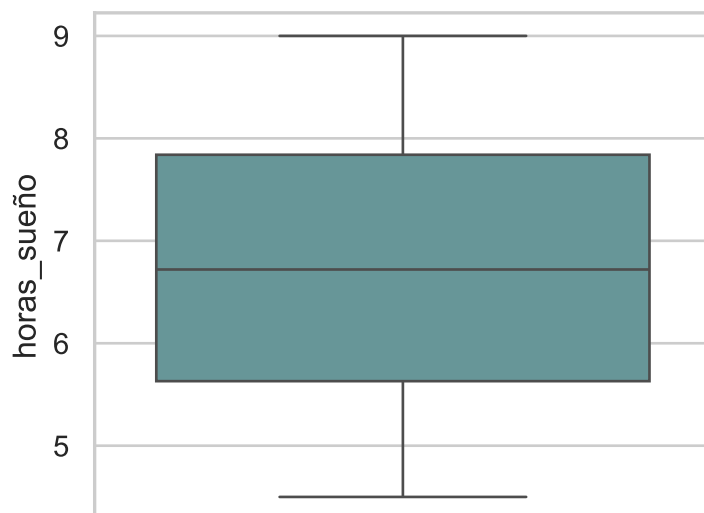
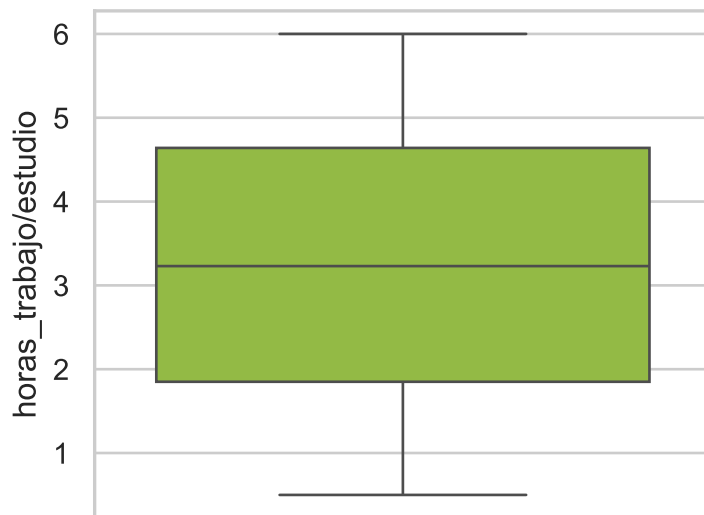
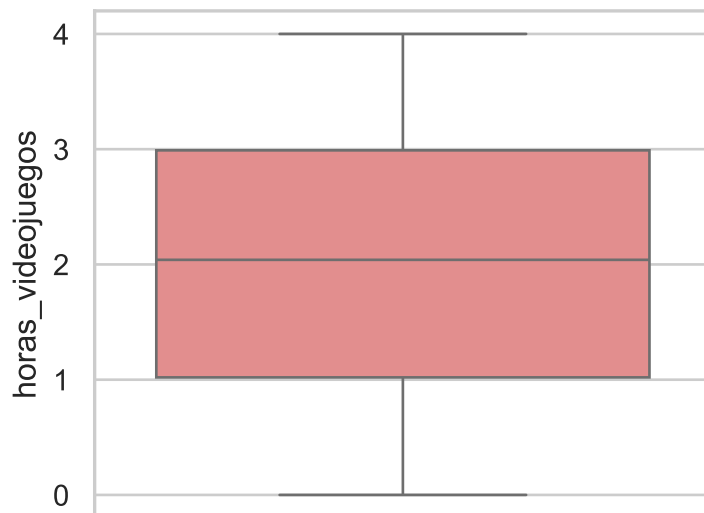
*Instrucciones para esta sección: para cada histograma, describir la forma de la distribución (simétrica, sesgada, con varias modas) y qué implica para el análisis (por ejemplo, si conviene una transformación o estandarización más adelante). Comentar si alguna variable de conteo tiene una escala muy distinta a las variables medidas en horas.*

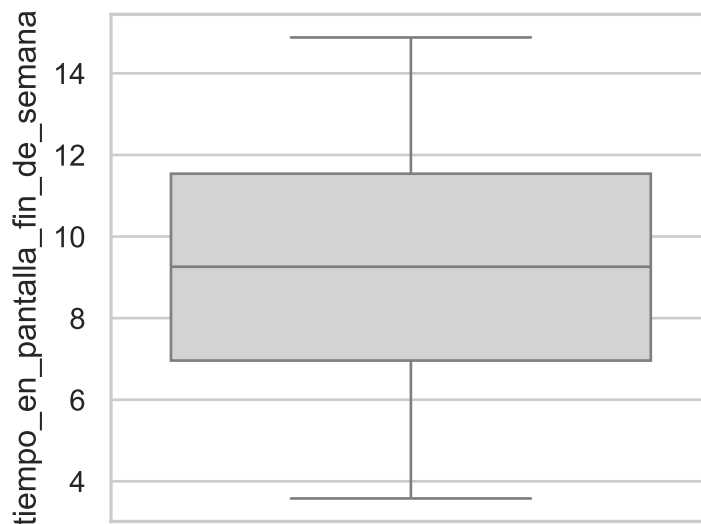
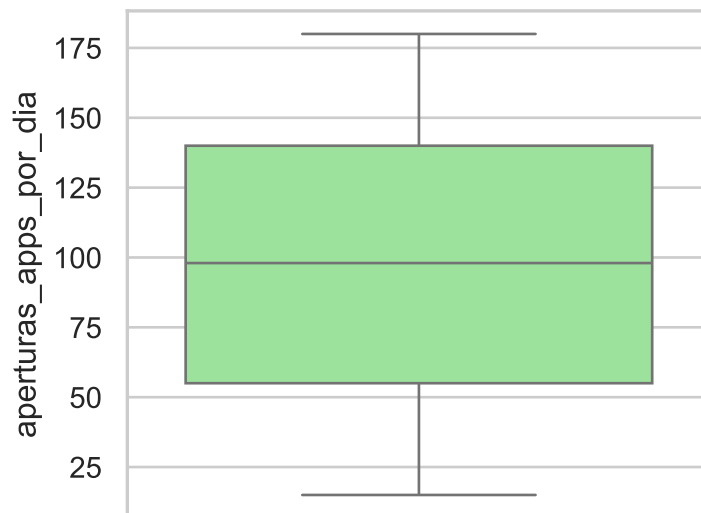
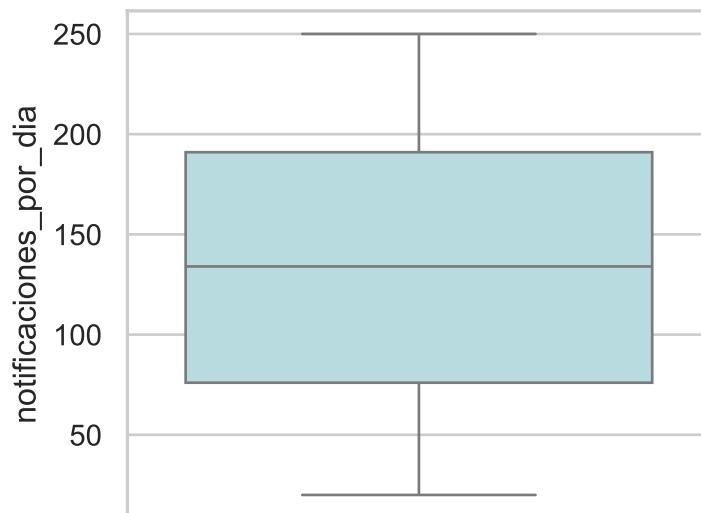
```
# TODO: graficar un boxplot por cada variable numérica (usar num_cols).
# asegúrate de que num_cols existe y tiene nombres reales

colores = ['steelblue', 'darkorange', 'forestgreen', 'lightcoral', 'yellowgreen', 'cadetblue',
           'powderblue', 'lightgreen', 'lightgray']
num_cols = [
    "edad",
    "horas_diarias_pantalla",
    "horas_redes_sociales",
    "horas_videojuegos",
    "horas_trabajo/estudio",
    "horas_sueño",
    "notificaciones_por_dia",
    "aperturas_apps_por_dia",
    "tiempo_en_pantalla_fin_de_semana",
] #aquí descartamos "etiqueta_de_adiccion"

contador = 0
for col in num_cols:
    plt.figure(figsize=(4, 3))
    sns.boxplot(
        data=df_clean,
        y=col,
        color=colores[contador],
    )
    contador += 1
    plt.xlabel("")
    plt.ylabel(col)
    plt.tight_layout()
    plt.show()
```







*Instrucciones para esta sección: comentar, variable por variable, la dispersión, la posible presencia de outliers y la simetría observada en los boxplots. Discutir si esos valores extremos deberían tratarse como errores de captura o si son observaciones reales del comportamiento de los usuarios, y justificar la decisión tomada.*

```
# TODO: calcular media, mediana, moda, desviación estándar, varianza, rango y cuartiles p
for i in num_cols:
    display(df_clean[i].describe())
```

```
count      7500.000000
mean        26.568800
std         5.197108
min         18.000000
25%         22.000000
50%         27.000000
75%         31.000000
max         35.000000
Name: edad, dtype: float64
```

```
count      7500.000000
mean         7.499912
std          2.609188
min          3.000000
25%          5.220000
50%          7.525000
75%          9.810000
max          12.000000
Name: horas_diarias_pantalla, dtype: float64
```

```
count      7500.000000
mean         3.273484
std           1.585342
min           0.500000
25%           1.910000
50%           3.270000
75%           4.630000
max           6.000000
Name: horas_redes_sociales, dtype: float64
```

```
count      7500.000000
mean         2.014183
std           1.146039
min           0.000000
25%           1.020000
50%           2.040000
75%           2.990000
max           4.000000
Name: horas_videojuegos, dtype: float64
```

```
count    7500.000000
mean      3.242420
std       1.600765
min       0.500000
25%      1.850000
50%      3.230000
75%      4.640000
max       6.000000
```

Name: horas\_trabajo/estudio, dtype: float64

```
count    7500.000000
mean      6.737561
std       1.283605
min       4.500000
25%      5.630000
50%      6.720000
75%      7.840000
max       9.000000
```

Name: horas\_sueño, dtype: float64

```
count    7500.000000
mean     134.257333
std      66.586883
min      20.000000
25%     76.000000
50%    134.000000
75%    191.000000
max     250.000000
```

Name: notificaciones\_por\_dia, dtype: float64

```
count    7500.000000
mean      97.832400
std      48.423349
min      15.000000
25%     55.000000
50%     98.000000
75%    140.000000
max     180.000000
```

Name: aperturas\_apps\_por\_dia, dtype: float64

```
count    7500.000000
mean      9.243827
std       2.718281
min       3.580000
```

|     |           |
|-----|-----------|
| 25% | 6.960000  |
| 50% | 9.260000  |
| 75% | 11.540000 |
| max | 14.880000 |

Name: tiempo\_en\_pantalla\_fin\_de\_semana, dtype: float64

*Instrucciones para esta sección: comparar media y mediana para detectar asimetría en cada variable, e interpretar la desviación estándar en el contexto de la escala de cada una (no comparar directamente variables con unidades distintas). Señalar qué variables muestran mayor dispersión o valores extremos.*

```
# TODO: graficar la frecuencia de las variables categóricas (cat_cols: genero, nivel_estr
for i in cat_cols:
    df_clean[i].value_counts(normalize=True)
```

genero

|        |          |
|--------|----------|
| Male   | 0.340400 |
| Other  | 0.331467 |
| Female | 0.328133 |

Name: proportion, dtype: float64

nivel\_estres

|        |          |
|--------|----------|
| High   | 0.341333 |
| Low    | 0.333733 |
| Medium | 0.324933 |

Name: proportion, dtype: float64

impacto\_academico/laboral

|     |        |
|-----|--------|
| No  | 0.5004 |
| Yes | 0.4996 |

Name: proportion, dtype: float64

estado\_adiccion

|           |          |
|-----------|----------|
| Adicto    | 0.707733 |
| No adicto | 0.292267 |

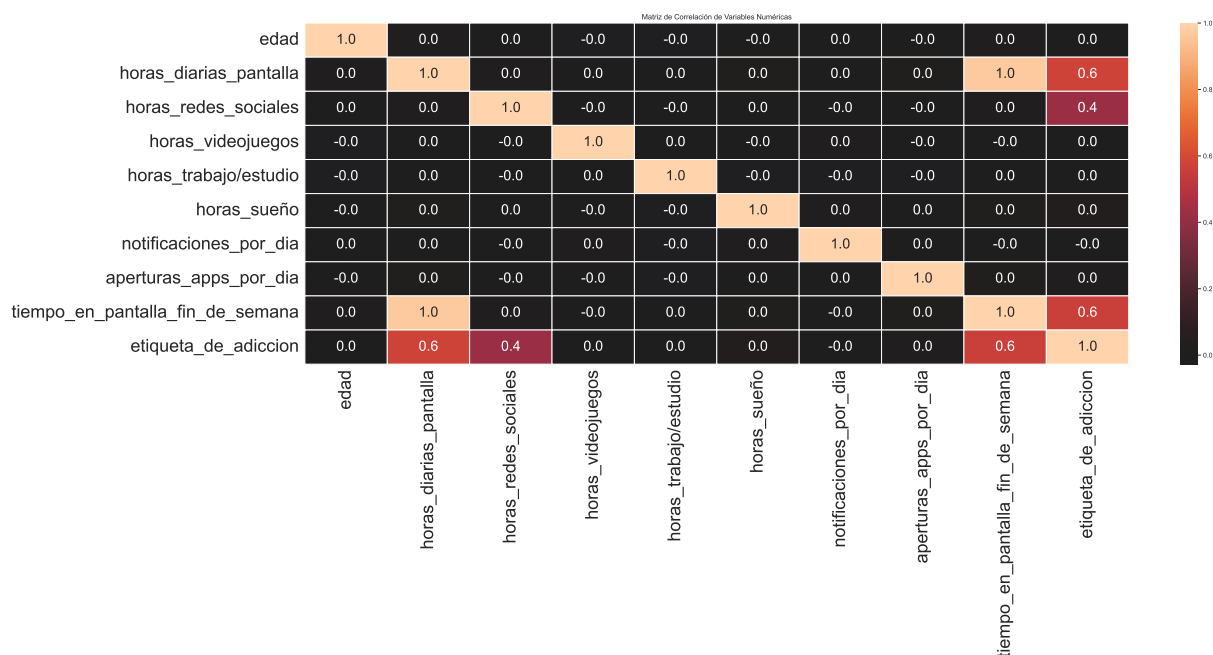
Name: proportion, dtype: float64

*Instrucciones para esta sección: describir qué categorías son dominantes o poco frecuentes en cada variable categórica y comentar si eso podría afectar el análisis posterior (por ejemplo, categorías con muy pocos casos). Indicar si alguna de estas variables se tratará como ordinal y por qué.*

## 6.2. Análisis multivariado

```
# TODO: graficar un mapa de calor de correlaciones entre las variables numéricas (num_col
df_num = df_clean[num_cols+ ["etiqueta_de_adiccion"]].copy()
# 1. Primero va el tamaño y la resolución (el lienzo)
plt.figure(figsize=(30, 10), dpi=300)
```

```
# 2. Luego armamos el mapa de calor con tus mismos parámetros
sns.heatmap(
    df_num.corr(),
    annot=True,
    fmt=".1f",
    vmax=1,
    center=0,
    linewidths=0.5, # Líneas divisorias sutiles
    cbar_kws={"shrink": 1}, # Barra de color un poco más chica
    annot_kws={"size": 23}, # Números chiquitos para que no se amontonen
)
plt.xticks(fontsize=29) # etiquetas del eje X más grandes
plt.yticks(fontsize=29) # etiquetas del eje Y más grandes
# 3. Ponemos el título y mostramos
plt.title("Matriz de Correlación de Variables Numéricas")
plt.show()
```

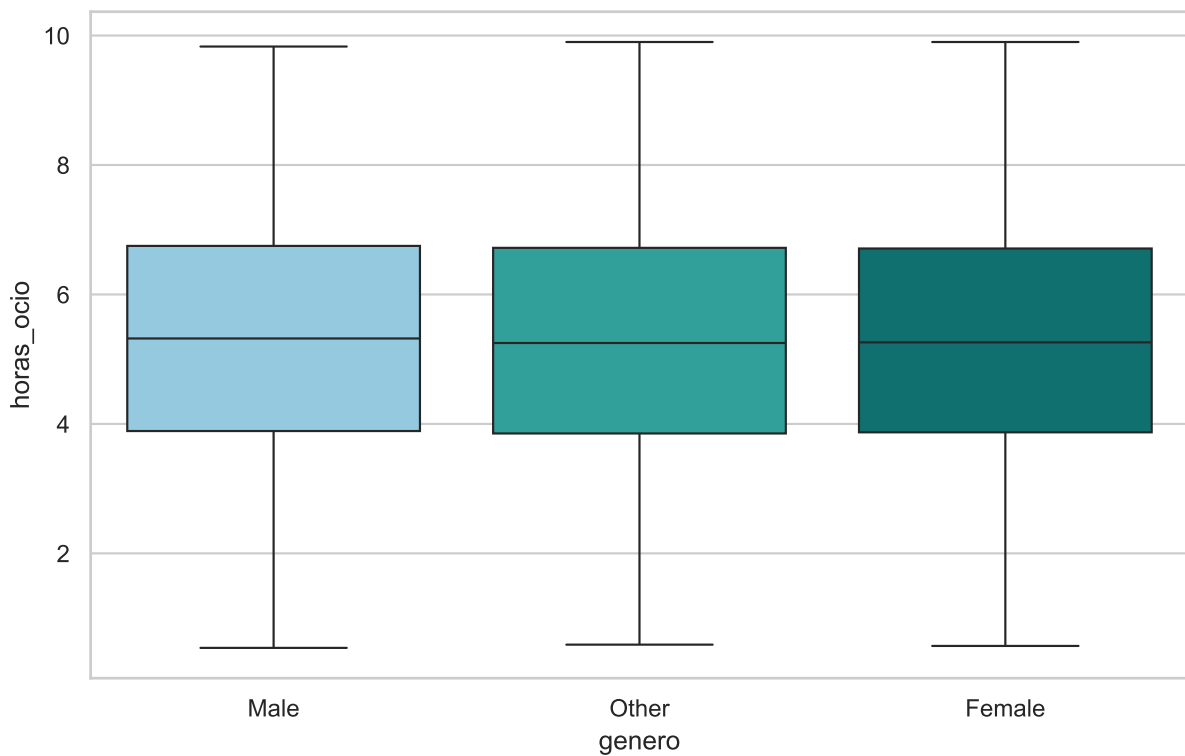
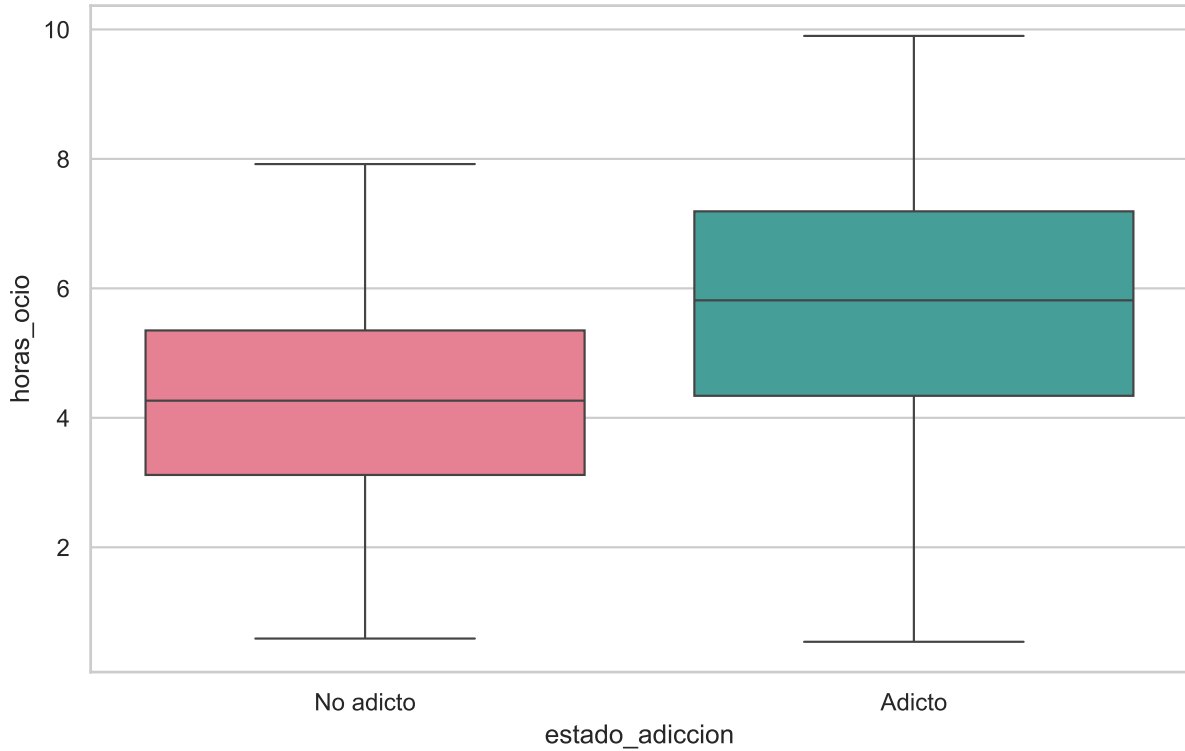


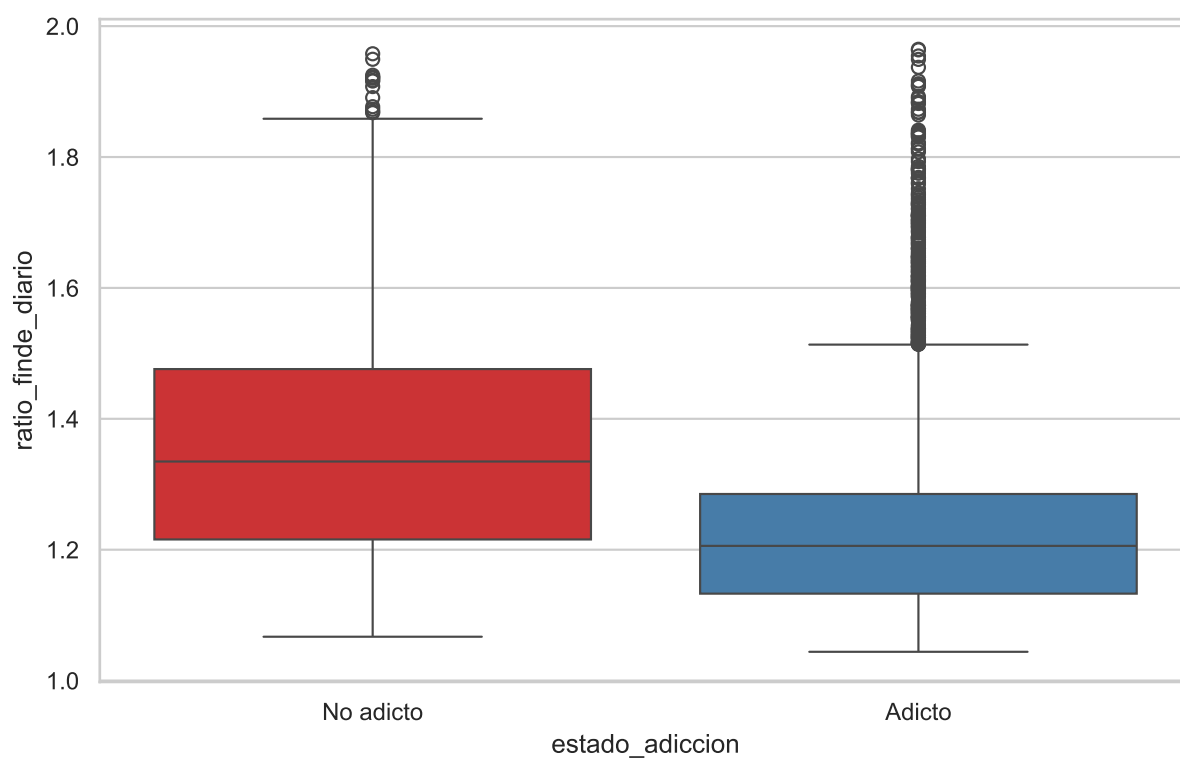
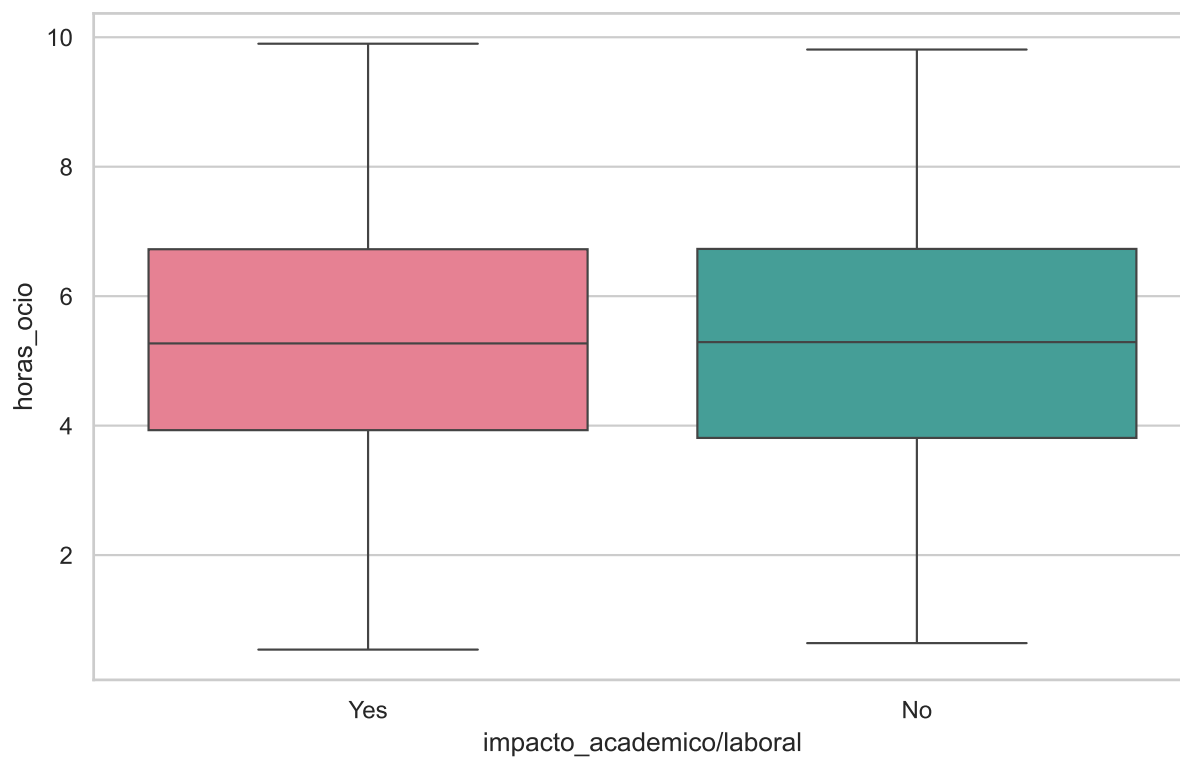
*Instrucciones para esta sección: identificar en el mapa de calor los pares de variables con correlación fuerte (positiva o negativa) y comentar si sugieren colinealidad. Aclarar que la correlación no implica causalidad.*

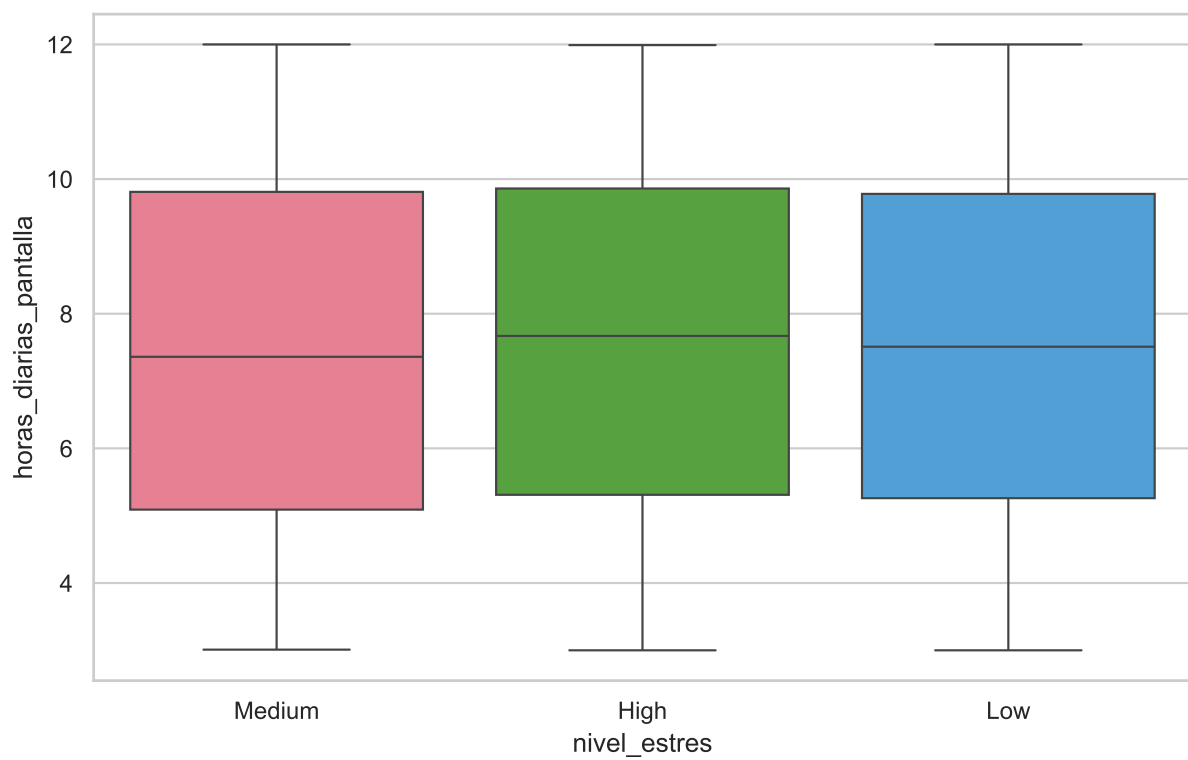
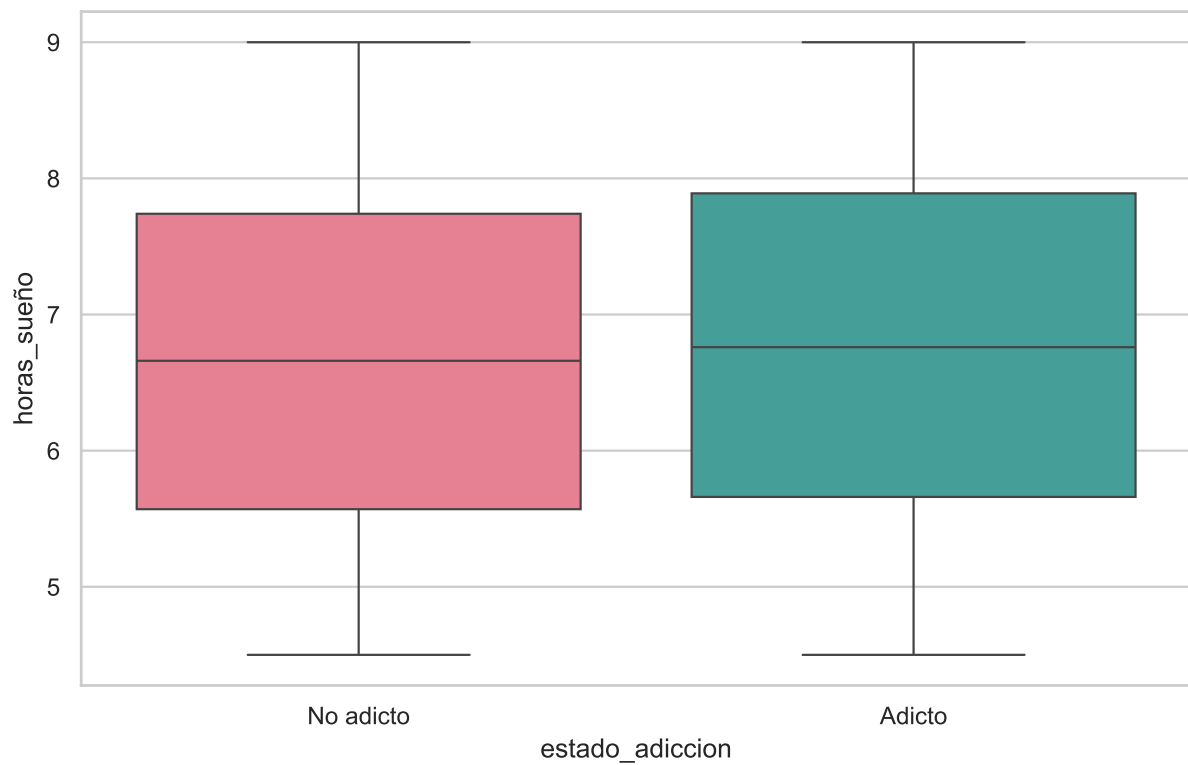
```
# TODO: calcular y ordenar las correlaciones de cada variable numérica (num_cols) con eti
#Chamba Jossep
```

*Instrucciones para esta sección: comentar qué variables tienen mayor asociación lineal con la etiqueta objetivo y cuáles tienen asociación muy baja. Aclarar que esto es solo un indicio y no una selección final de variables para el modelo.*

```
# TODO: graficar boxplots comparativos de variables numéricas clave (por ejemplo, horas_d
del plt
import matplotlib.pyplot as plt
sns.boxplot(data=df_clean, x="estado_adiccion", y="horas_ocio", palette = "husl", legend
```



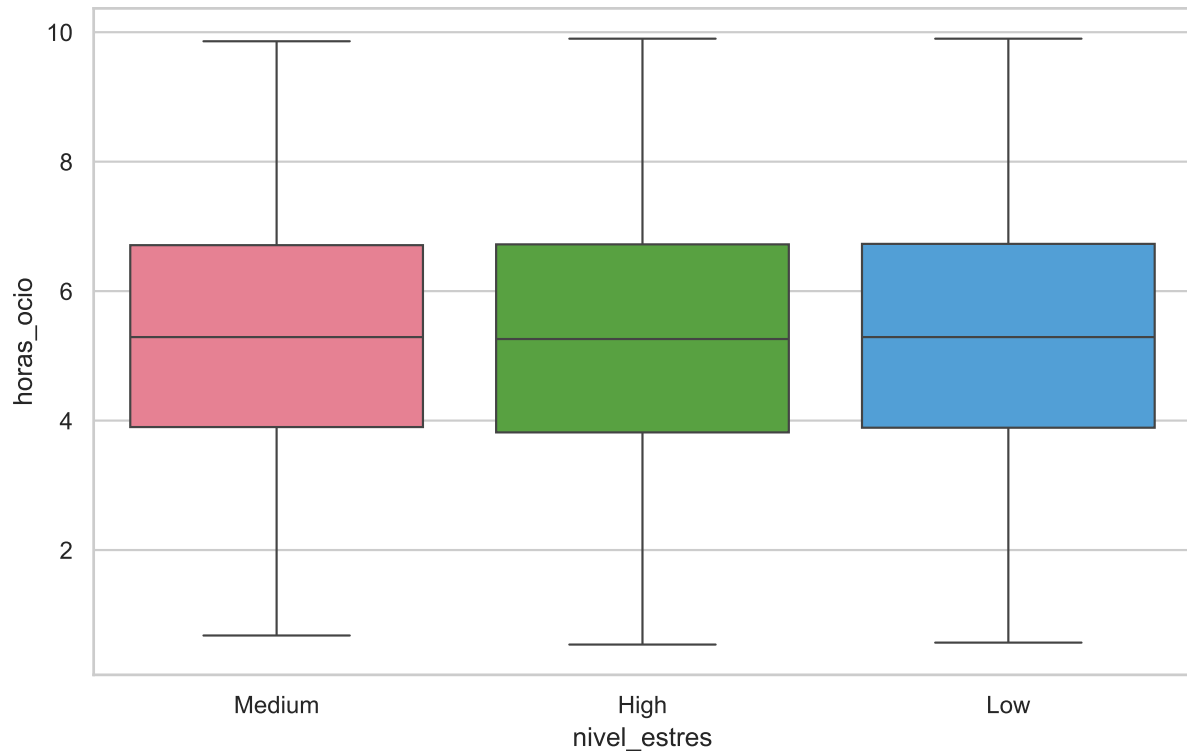




```
[ 'edad',  
  'horas_diaras_pantalla',  
  'horas_redes_sociales',  
  'horas_videojuegos',  
  'horas_trabajo/estudio',  
  'horas_sueño',
```

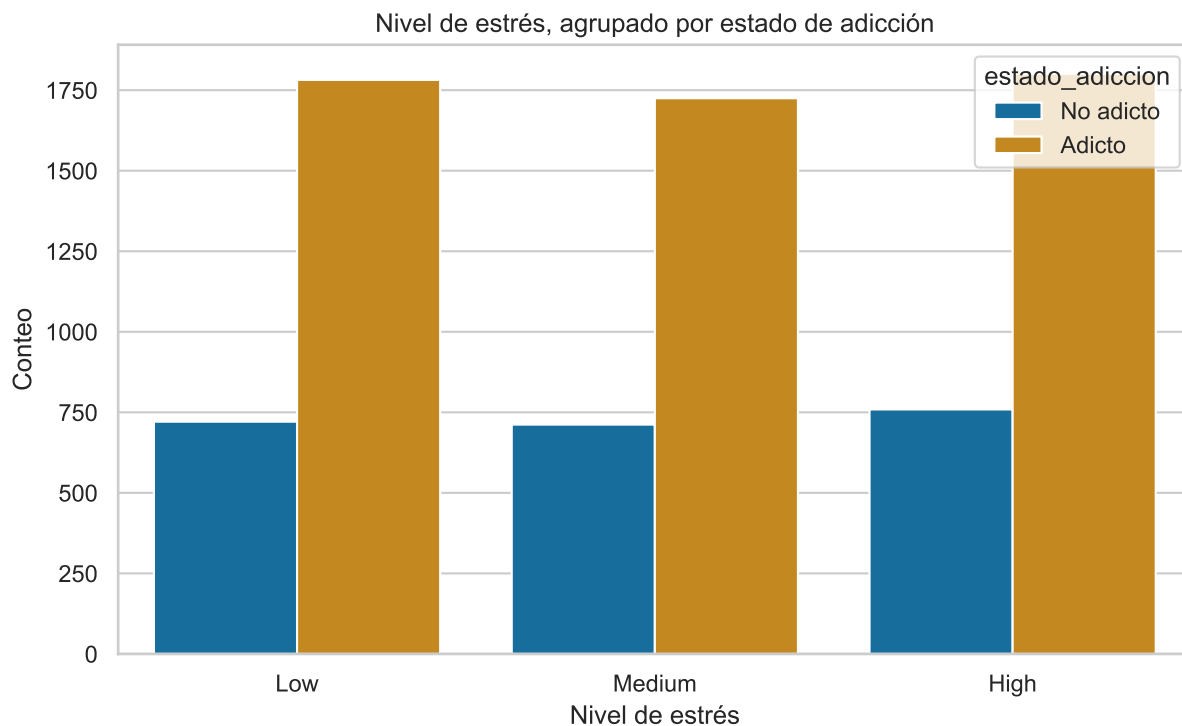
```
'notificaciones_por_dia',  
'aperturas_apps_por_dia',  
'tiempo_en_pantalla_fin_de_semana']
```

```
['genero', 'nivel_estres', 'impacto_academico/laboral', 'estado_adiccion']
```

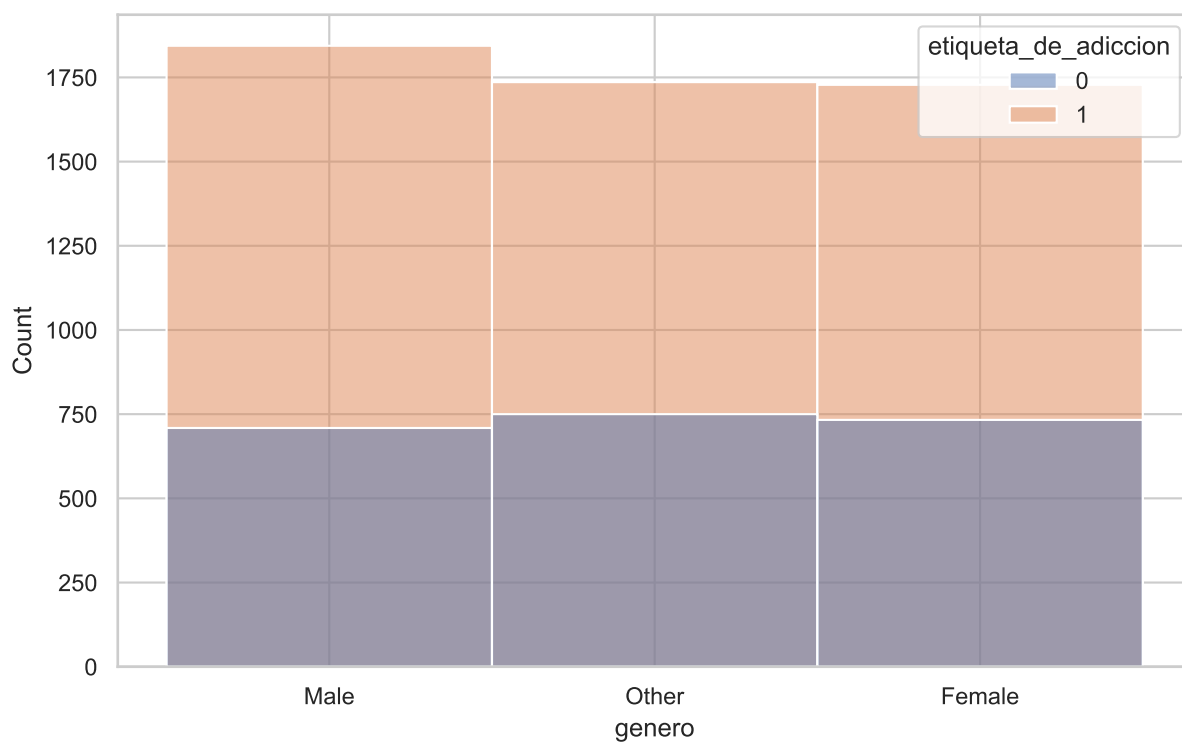


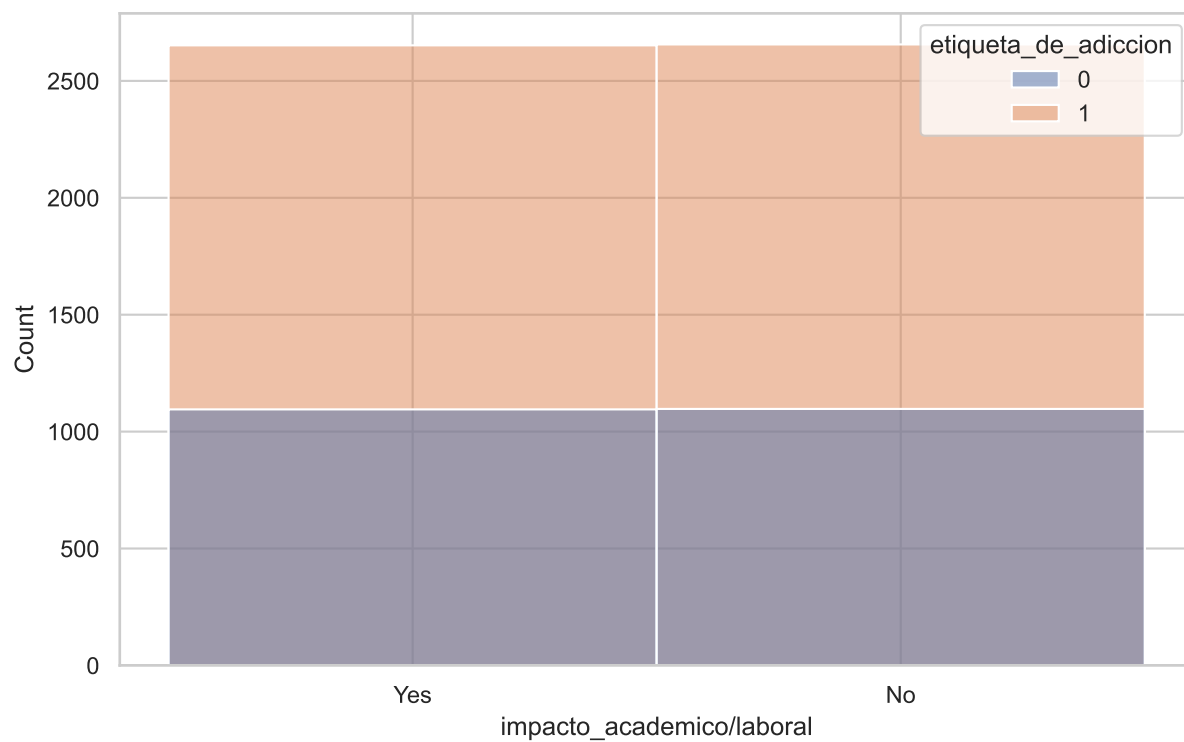
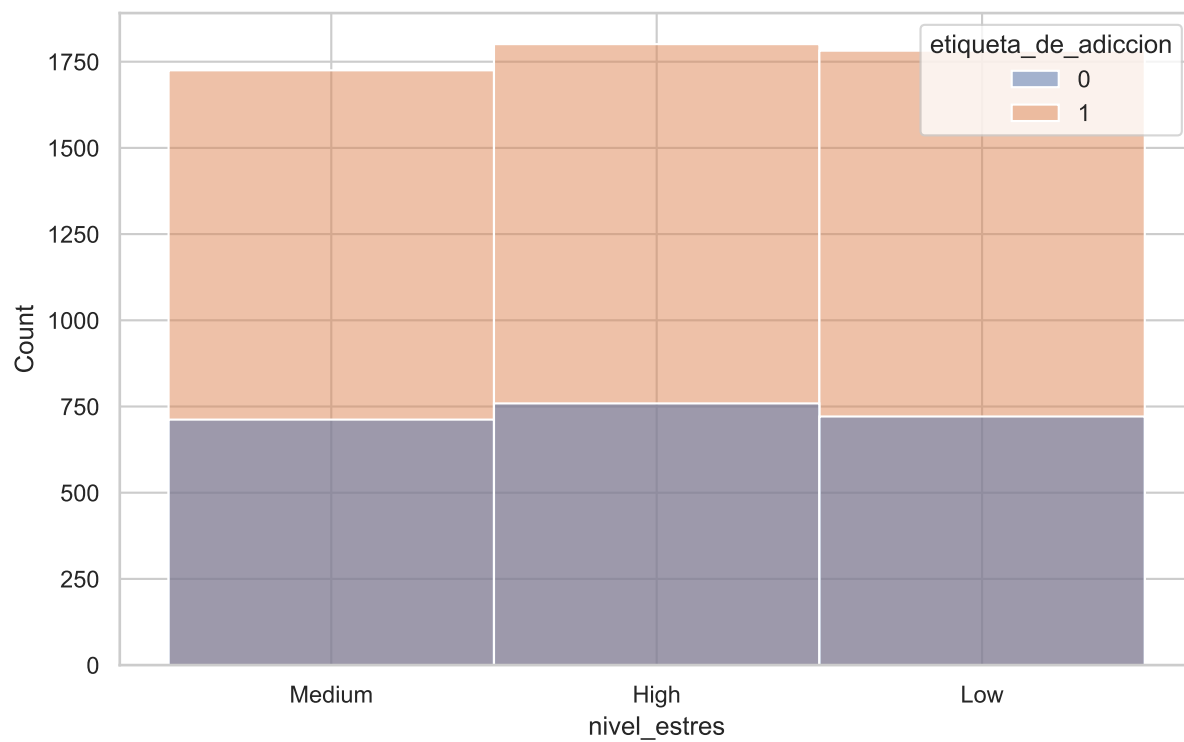
*Instrucciones para esta sección: comentar cómo cambian la tendencia central y la dispersión de cada variable entre las dos clases, y si hay solapamiento amplio entre grupos (lo que anticiparía dificultad para separar las clases con un solo predictor).*

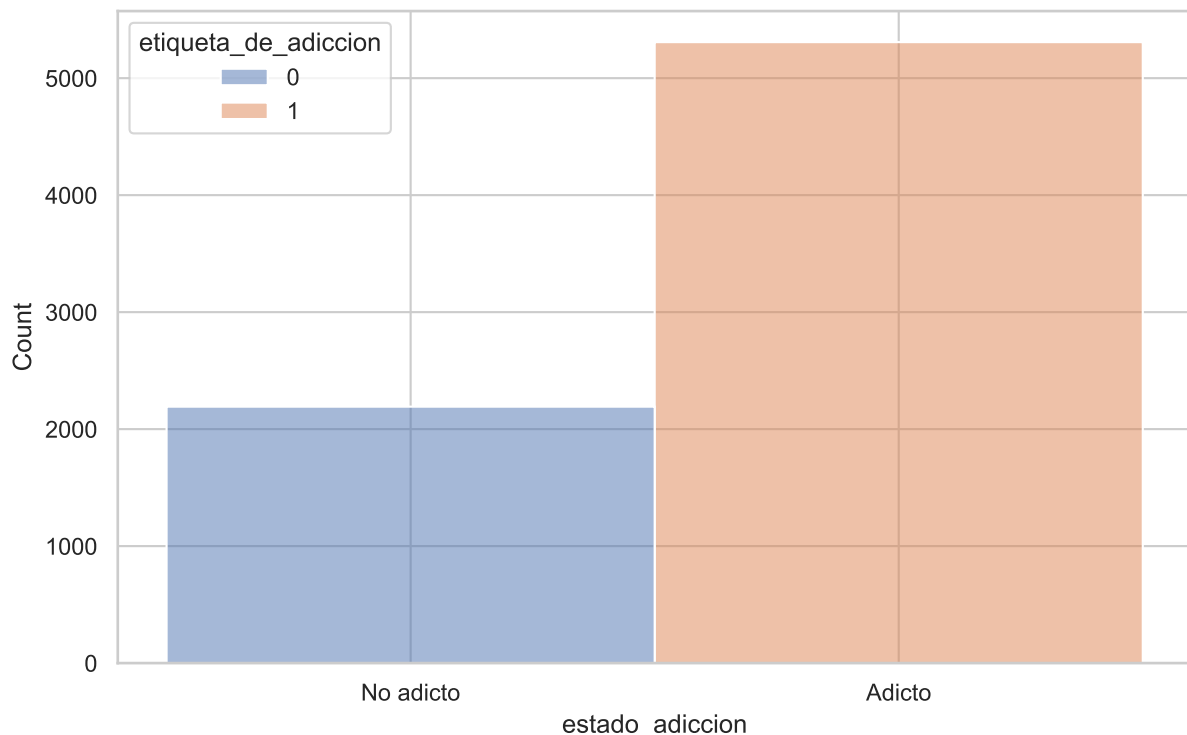
```
plt.figure(figsize=(8,5))  
sns.countplot(data=df_clean, x="nivel_estres", hue="estado_adiccion",  
              order=["Low","Medium","High"], palette="colorblind")  
plt.title("Nivel de estrés, agrupado por estado de adicción")  
plt.xlabel("Nivel de estrés")  
plt.ylabel("Conteo")  
plt.tight_layout()  
plt.show()
```



```
# TODO: graficar histogramas o barras agrupadas comparando subgrupos (por ejemplo, nivel_
for columna in cat_cols:
    sns.histplot(data=df_clean, x = columna, hue = 'etiqueta_de_adiccion')
    plt.show()
```







*Instrucciones para esta sección: interpretar las proporciones (no solo los conteos) entre subgrupos, evitando conclusiones que estigmaticen a los usuarios. Comentar si estas variables parecen ser causa, consecuencia o simplemente indicadores asociados al uso problemático.*

# TODO: graficar al menos 5 diagramas de dispersión (scatterplots) entre pares de variables.

# TODO: graficar al menos 5 diagramas de dispersión (scatterplots) entre pares de variables.

*Instrucciones para esta sección: para cada par de variables, comentar si se observa un patrón (lineal o no lineal), agrupamientos, o separación visible entre clases. Si los colores se solapan mucho, señalar que esas dos variables por sí solas no bastan para separar las clases.*

# TODO: calcular la covarianza entre pares de variables numéricas relevantes (num\_cols).

*Instrucciones para esta sección: interpretar el signo de la covarianza (si dos variables crecen o decrecen juntas) y explicar por qué se complementa con la correlación (que sí permite comparar la fuerza de la relación entre variables con escalas distintas).*

## 7. Conclusiones de P1

*Instrucciones para esta sección: redactar entre 4 y 6 conclusiones numeradas, basadas únicamente en los resultados obtenidos arriba (no inventar cifras). Deben cubrir, como mínimo: (1) calidad de datos (nulos, duplicados) y qué se hizo al respecto, (2) si la clase objetivo está balanceada o no y qué implica para P2, (3) qué variables se excluyen como predictoras y por qué, (4) qué variables lucen más prometedoras para el modelamiento, y (5) qué se debe tener en cuenta al pasar a la P2 (codificación, escalamiento, evaluación).*

## 8. Limitaciones y consideraciones éticas

El archivo no incluye documentación sobre el muestreo, instrumento de medición, país, periodo de recolección ni procedimiento de etiquetado. En consecuencia, la muestra no debe asumirse representativa de la población universitaria peruana ni de un grupo demográfico específico. Además, el término “adicción” debe tratarse como una etiqueta del dataset y no como un diagnóstico clínico. La evidencia académica recomienda emplear instrumentos validados y distinguir entre uso frecuente y uso problemático Kwon et al. (2013); Csibi et al. (2018).

Si este análisis se trasladara a un contexto real, la privacidad, el consentimiento informado y el riesgo de estigmatización serían centrales. Una predicción debe apoyar la prevención o la derivación voluntaria, nunca justificar sanciones automáticas o decisiones de alto impacto.

## Referencias

- Amez, S y S Baert (2020). Smartphone Use and Academic Performance: A Literature Review. *International Journal of Educational Research* **103**, 101618.
- Csibi, S, MD Griffiths, B Cook, Z Demetrovics y A Szabo (2018). The Psychometric Properties of the Smartphone Application-Based Addiction Scale (SABAS). *International Journal of Mental Health and Addiction* **16**(2), 393-403.
- Kwon, M, JY Lee, WY Won, JW Park, JA Min, C Hahn, X Gu, JH Choi y DJ Kim (2013). Development and Validation of a Smartphone Addiction Scale (SAS). *PLOS ONE* **8**(2), e56936.
- Nusrat, Z (s.f.). *Smartphone Usage and Addiction Analysis Dataset*. Kaggle. Fecha de publicación no indicada en la página; consultado el 17 de septiembre de 2026. <https://www.kaggle.com/datasets/zahranusratt/smartphone-usage-and-addiction-analysis-dataset>.
- Pedregosa, F, G Varoquaux, A Gramfort, V Michel, B Thirion, O Grisel, M Blondel, P Prettenhofer, R Weiss, V Dubourg, J Vanderplas, A Passos, D Cournapeau, M Brucher, M Perrot y É Duchesnay (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* **12**, 2825-2830.